

RAG (RetrievalAugmented Generation)

Prof : Dr Mohammed BOUSMAH

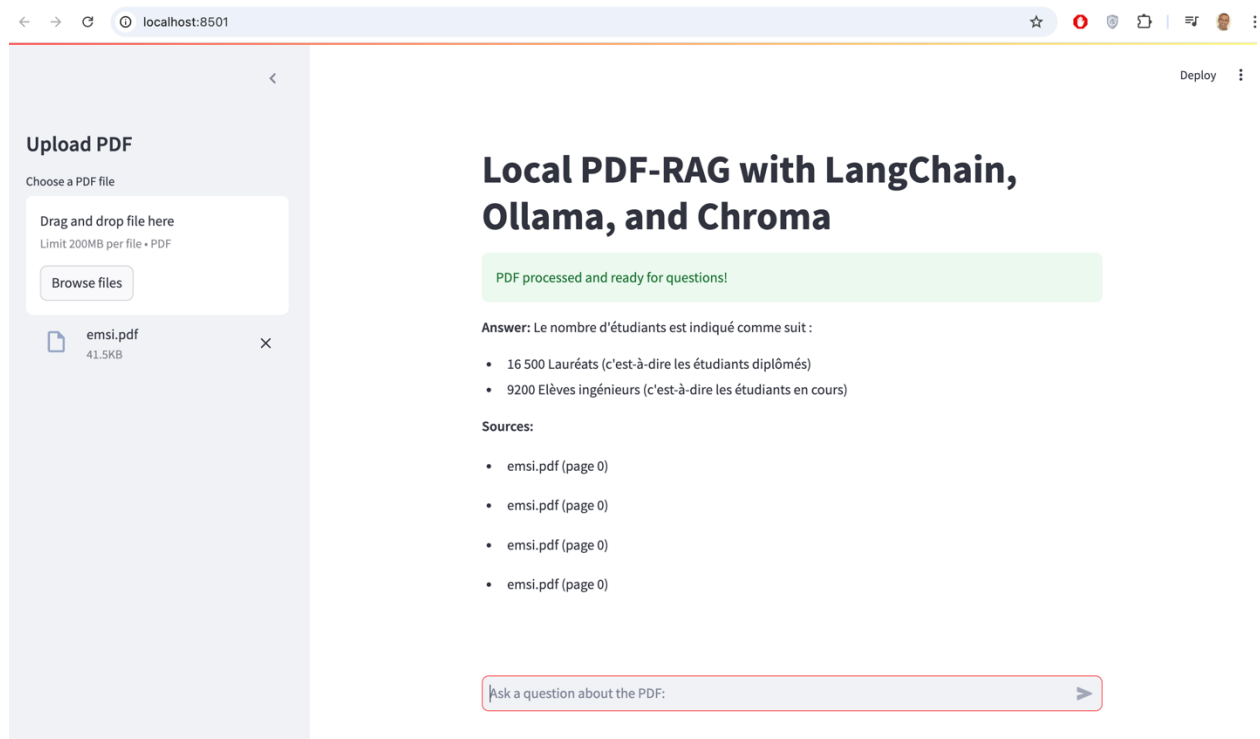
But: Mise en place d'un RAG EMSI-ChatBot

Résumé Fonctionnel

1. L'utilisateur upload un fichier PDF.
2. Le fichier est analysé et transformé en une base de données vectorielle.
3. L'utilisateur pose des questions sur le contenu du PDF.
4. Le programme génère des réponses basées sur les informations pertinentes du fichier PDF, en citant les sources.

T.A.F:

- 1- Démarrer le programme
- 2- Analyser le code en proposant un schéma expliquant l'architecture technique du RAG
- 3- Améliorer le programme



1. Principe de Base

- **Retrieval (Récupération) :**

Avant de répondre à une question, le système récupère des informations pertinentes depuis une base de données, un ensemble de documents ou une autre source d'informations.

- **Augmented Generation (Génération Augmentée) :**

Un modèle de langage génératif, comme GPT ou un modèle similaire, utilise les informations récupérées pour générer une réponse cohérente et enrichie, au lieu de s'appuyer uniquement sur sa propre base de connaissances.

2. Avantages de RAG

- **Réponses contextuelles basées sur des données externes :**

Contrairement à un modèle génératif seul, RAG garantit que les réponses sont liées à des informations spécifiques, par exemple des articles, des bases de données ou des PDF.

- **Moins de désinformation :**

Puisque le modèle se base sur des documents précis, il est moins susceptible d'inventer des réponses.

- **Mises à jour dynamiques :**

Les données étant externes, les réponses peuvent être ajustées sans avoir besoin de réentraîner le modèle.

3. Fonctionnement Technique

Le flux d'une application RAG typique se déroule comme suit :

1. **Indexation des Documents :**

- Les documents sont analysés et transformés en vecteurs (représentations numériques) à l'aide d'outils comme **HuggingFaceEmbeddings**, **OpenAI Embeddings**, ou d'autres encodeurs.
- Ces vecteurs sont stockés dans une base de données vectorielle comme **Chroma**, **FAISS**, ou **Weaviate**.

2. **Récupération d'Informations Pertinentes :**

- Une question (ou requête) est posée.
- La requête est transformée en vecteur, puis comparée aux documents indexés pour récupérer les plus pertinents (selon une métrique comme la similarité cosinus).

3. **Génération Augmentée :**

- Les documents récupérés sont transmis à un modèle de langage (LLM, Large Language Model), qui utilise ces informations pour générer une réponse.
- La chaîne d'outils comme **RetrievalQA** orchestre cette étape.

4. Limitations et Défis

- **Qualité des données** : Les réponses dépendent fortement de la qualité et de la pertinence des documents indexés.
- **Performance** : Le processus de récupération et de génération peut être lent pour de grands ensembles de données.
- **Maintenance des données** : Les bases de données doivent être régulièrement mises à jour pour rester pertinentes.

```
#####
# But: Mise en place d'un RAG EMSI-ChatBot
# T.A.F: 1/ Démarrer le programme 2/ Analyser le code 3/
Améliorer le programme
#####

import streamlit as st
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import
RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import Ollama
from langchain.chains import RetrievalQA

# Set Streamlit page configuration
st.set_page_config(page_title="PDF-RAG", page_icon="📄")

# Sidebar for PDF upload
st.sidebar.title("Upload PDF")
uploaded_file = st.sidebar.file_uploader("Choose a PDF file",
type="pdf")

# Main area for Q&A
st.title("Local PDF-RAG with LangChain, Ollama, and Chroma")

# Initialize session state
```

```

if "db" not in st.session_state:
    st.session_state.db = None

if "qa_chain" not in st.session_state:
    st.session_state.qa_chain = None

# Handle PDF upload and processing
if uploaded_file is not None:
    with st.spinner("Processing PDF..."):
        # Save the uploaded file temporarily
        with open(uploaded_file.name, "wb") as f:
            f.write(uploaded_file.getbuffer())

        # Load and split the PDF
        loader = PyPDFLoader(uploaded_file.name)
        documents = loader.load()
        text_splitter =
RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=150)
        texts = text_splitter.split_documents(documents)

        # Create embeddings and store in Chroma
        embeddings = HuggingFaceEmbeddings()
        st.session_state.db = Chroma.from_documents(texts,
embeddings)

        # Initialize the QA chain with Ollama
        llm = Ollama(model="llama3.2") # Or your preferred
Ollama model
        st.session_state.qa_chain = RetrievalQA.from_chain_type(
            llm=llm,
            chain_type="stuff",
            retriever=st.session_state.db.as_retriever(),
            return_source_documents=True,
        )

```

```

        st.success("PDF processed and ready for questions!")

# User input for query
query = st.chat_input("Ask a question about the PDF:")

# Generate and display response
if query and st.session_state.qa_chain:
    with st.spinner("Generating response..."):
        result = st.session_state.qa_chain(query)
        st.write("**Answer:**", result["result"])
        st.write("**Sources:**")
        for doc in result["source_documents"]:
            st.write(f"- {doc.metadata['source']} (page {doc.metadata['page']})")

```

Ce programme est une application Streamlit qui permet de poser des questions sur le contenu d'un fichier PDF en utilisant des outils d'intelligence artificielle (LangChain, Ollama, et Chroma). Voici une explication détaillée :

1. Bibliothèques Importées

- **streamlit** : Crée une interface utilisateur simple et interactive.
- **langchain** : Fournit des outils pour traiter des documents et des chaînes AI.
 - **PyPDFLoader** : Charge le contenu des fichiers PDF.
 - **RecursiveCharacterTextSplitter** : Divise les documents en segments plus petits.
 - **HuggingFaceEmbeddings** : Génère des représentations vectorielles des textes pour les recherches.
 - **Chroma** : Stocke les représentations vectorielles dans une base de données.
 - **Ollama** : Fournit un modèle de langage pour répondre aux questions.
 - **RetrievalQA** : Crée une chaîne de questions-réponses en utilisant un récupérateur de documents.

2. Configuration Streamlit

- **Page Configuration :**

Configure l'apparence et le titre de l'application :

python

Copier le code

```
st.set_page_config(page_title="PDF-RAG", page_icon="📄")
```

- **Barre latérale :**

Permet de téléverser un fichier PDF via :

python

Copier le code

```
uploaded_file = st.sidebar.file_uploader("Choose a PDF file", type="pdf")
```

3. Gestion de l'État de Session

- L'état de session conserve des données sur la base de données vectorielle et la chaîne QA, pour qu'elles soient disponibles tout au long de l'interaction :

python

Copier le code

```
if "db" not in st.session_state:
    st.session_state.db = None
if "qa_chain" not in st.session_state:
    st.session_state.qa_chain = None
```

4. Traitement du Fichier PDF

Lorsqu'un fichier est téléversé :

1. Enregistrement temporaire du fichier :

python

Copier le code

```
with open(uploaded_file.name, "wb") as f:
    f.write(uploaded_file.getbuffer())
```

2. Chargement et segmentation :

Le PDF est chargé avec PyPDFLoader, et son contenu est segmenté en morceaux de texte avec RecursiveCharacterTextSplitter. Les paramètres `chunk_size=1000` et `chunk_overlap=150` définissent la taille des segments et le chevauchement.

3. Création d'embeddings :

Le texte segmenté est converti en vecteurs avec HuggingFaceEmbeddings, et les vecteurs sont stockés dans une base de données vectorielle avec Chroma :

python

Copier le code

```
embeddings = HuggingFaceEmbeddings()
st.session_state.db = Chroma.from_documents(texts, embeddings)
```

4. Chaîne de questions-réponses (QA) :

Un modèle Ollama (comme "llama3.2") est utilisé pour répondre aux questions en récupérant les documents pertinents depuis la base vectorielle :

python

Copier le code

```
llm = Ollama(model="llama3.2")
st.session_state.qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=st.session_state.db.as_retriever(),
    return_source_documents=True,
)
```

5. Interaction Utilisateur

- **Saisie de questions :**

L'utilisateur pose une question via une interface interactive :

python

Copier le code

```
query = st.chat_input("Ask a question about the PDF:")
```

- **Génération et affichage des réponses :**

La chaîne QA génère une réponse en récupérant les documents pertinents, puis affiche :

- La réponse.
- Les sources avec les pages :

python

Copier le code

```
result = st.session_state.qa_chain(query)
```

```
st.write("**Answer:**", result["result"])
```

```
st.write("**Sources:**")
```

```
for doc in result["source_documents"]:
```

```
    st.write(f"- {doc.metadata['source']} (page {doc.metadata['page']})")
```
